

Week 4 Flight Data Pipeline & Multi-Domain Streaming Architecture

Objective

Extend the real-time streaming pipeline built in Week 3 (weather data via Kafka, PostgreSQL, and Airflow) to a second, independent data domain live flight tracking proving the architecture could support multiple concurrent data sources sharing the same infrastructure.

Work Performed

1. Flight Data Ingestion

Built `flight_producer.py`, which integrates the AeroDataBox flight-schedule API to poll scheduled arrivals and departures for a set of tracked airports (Lahore, Karachi, Islamabad). Since AeroDataBox returns every route touching a queried airport rather than only the routes of interest, a route filter (`TRACKED_ROUTE_PAIRS`) was implemented to drop all untracked/noisy routes before the data ever reaches Kafka.

2. Stream Processing

Built `flight_consumer.py` to consume flight events from the new `flight-raw` Kafka topic and upsert them into a PostgreSQL staging table, keeping the pipeline idempotent on re-runs.

3. Database Schema

Added `staging.flight_raw` to hold raw flight events, with a generated deduplication key combining scheduled time, airline, and route necessary because flight numbers are not globally unique across airlines, and a naive uniqueness check on timestamp alone silently missed duplicates. Also added `staging.api_usage_tracker` to monitor daily request/unit consumption against AeroDataBox's free-tier limits.

4. Data Warehouse

Extended the star-schema warehouse with `warehouse.dim_airport` and `warehouse.fact_flight_daily`, aggregating scheduled and cancelled flight counts per route, per day, per airline.

5. Orchestration

Built a second Airflow DAG, `flight_transformation_pipeline`, reusing the hourly `CronDataIntervalTimetable` pattern established in Week 3, to transform raw flight events into the warehouse layer on a scheduled cadence.

6. BI-Ready Views

Added Tableau-facing SQL views — `v_latest_flight_status`, `v_flight_route_summary`, and `v_api_usage_summary` — to support a planned "Flight Risk Advisor" dashboard.

Technologies Used

AeroDataBox API, Apache Kafka, PostgreSQL, Apache Airflow, Python (requests, psycpg2, kafka-python), Docker Compose, Git/GitHub (feature branch + pull request workflow).

Milestone Achieved

A second, independent streaming data domain (flights) was successfully added to the existing Kafka/Airflow/PostgreSQL architecture from Week 3, without disrupting the existing weather pipeline — validating that the architecture generalizes beyond a single data source. The work was submitted via a pull request ("Week 4") on GitHub for review.

Difficulties Faced and Solutions

The first major difficulty encountered was that AeroDataBox's airport-polling endpoint returns every flight touching a queried airport, not just the specific routes of interest. Left unfiltered, this would have flooded the pipeline with hundreds of irrelevant one-off routes. This was resolved by building a route-filtering mechanism that checks each incoming flight against a curated list of tracked route pairs, dropping anything outside that scope before it ever reaches Kafka.

A second, more subtle difficulty arose around duplicate data. Flight numbers are not globally unique across different airlines, so relying on the flight number alone as a uniqueness check was insufficient. Compounding this, scheduled departure time is often missing for records representing arrivals at the tracked airport, and since

databases treat two missing values as never equal to each other, a straightforward uniqueness constraint silently failed to catch duplicate records. This was solved by constructing a composite deduplication key that combines whichever scheduled time is available (departure or arrival), the airline, and the route, giving a stable identifier regardless of which side of the data the record originated from.

A third difficulty was that AeroDataBox's free tier imposes stricter usage limits than the weather API used in the previous phase, with no existing way to monitor consumption against those limits. To address this, a dedicated tracking table was introduced to record daily request and unit consumption, allowing usage to be monitored proactively rather than discovered only after hitting a rate limit.